Specifying Dynamical Systems for Neural CLF Training: Moving Towards User-Friendly Interfaces

Gil Benezer, Fang-Hsin “Cindy” Li

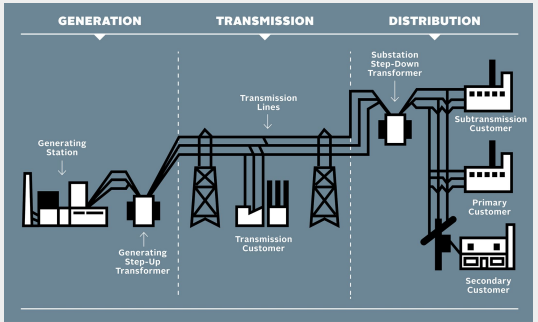
https://github.com/gbenezer/Lyapunov_Stable_NN_Controllers_Custom_Dynamics

Today's agenda

- Background: Controller Verification and CLFs
- Issue: An SotA Method for CLFs Isn't User Friendly
- Approach: Make a Better Interface
- Results: Does the Interface Perform Well?
- Conclusion: What We Did and Where To Next

Background: Neural Controllers

- Neural networks are increasingly proposed for control of safety-critical cyber-physical systems
  - Self-driving vehicles
  - Airplanes and UAVs
  - Medical devices
  - Power grids
  - Manufacturing processes

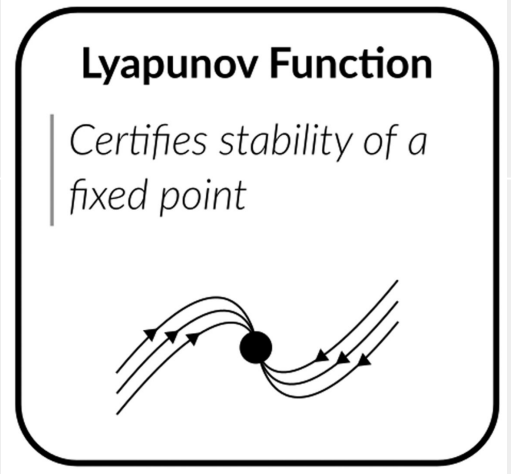


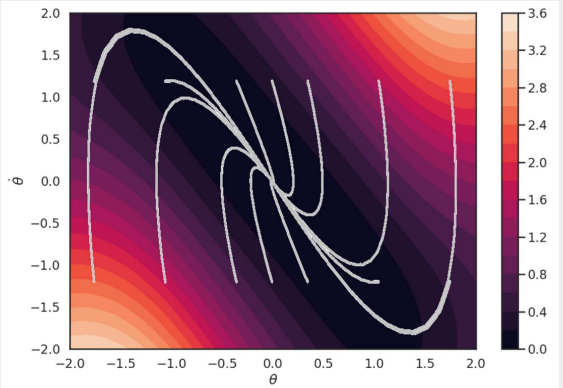
Background: Neural Controller Verification

- Three main approaches
  - Reachability Analysis
    - Provides bounds on states that can be reached given a dynamics model and control policy
  - Safe/Constrained RL
    - Learns a control policy without a dynamics model, with costs for departure from safety or explicit safety constraints
  - Certificate Functions
    - Verifies safety and/or stability properties around states or trajectories given a dynamical system and control policy

Background: CLFs

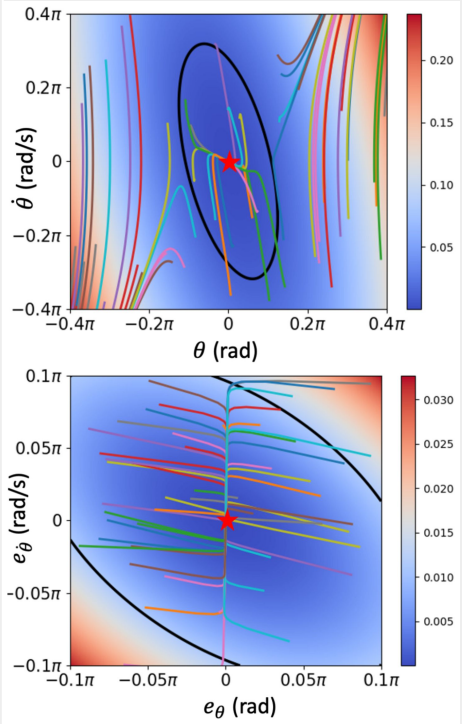
- Control Lyapunov Functions (CLF)
  - Certifies a system can be stabilized around a equilibrium point in state space
  - Roughly has to satisfy the following properties
    - Positive everywhere but equilibrium
    - Zero at equilibrium
    - Always decreasing
  - The area around the equilibrium point that is certified is known as the region-of-attraction (ROA)





Figures: *Safe Control With Learned Certificates: A Survey of Neural Lyapunov, Barrier, and Contraction Methods for Robotics and Control*
Charles Dawson , Sicun Gao, and Chuchu Fan

Background: SotA Method



- *Lyapunov-stable Neural Control for State and Output Feedback: A Novel Formulation* by Lujie Yang, Hongkai Dai, Zhouxing Shi, Cho-Jui Hsieh, Russ Tedrake, and Huan Zhang
  - Introduces novel method to verify and synthesize CLFs even with output feedback (state is not observable)
  - Uses α,β-CROWN for verification of constraints rather than MILP or SMT solvers to improve efficiency

Issue: Code Is Not User-Friendly

- Authors provide example scripts to reproduce some (but not all) experiments
  - One script doesn’t even do that by default and needs special configuration
- Scripts rely on hard-coded implementations of dynamical systems
- Visualizations are of the Lyapunov function for the open loop system
- No work was done regarding quantitative measurement ROA volumes

Approach: Dynamical System Interface

- Unified SymbolicDynamicalSystem superclass
  - Sub-classes define initialization and system definition methods
  - System definition by user constructs state, control, and output variables in terms of SymPy Symbols
  - User relates symbolic variables to define system state and output dynamics
- GenericDiscreteTimeSystem wrapper class
  - Interface for using the dynamical system with the CLF training code base

Approach: System Example

State-Space Formulation:

$$\frac{d}{dt}\begin{bmatrix}\theta\\ \dot{\theta}\end{bmatrix} = \begin{bmatrix}\dot{\theta}\\ \ddot{\theta}\end{bmatrix} = \begin{bmatrix}\dot{\theta}\\ \dfrac{mgl\sin(\theta) + u - b\dot{\theta}}{ml^2}\end{bmatrix} = \begin{bmatrix}\dot{\theta}\\ \dfrac{-b}{ml^2}\cdot\dot{\theta} + \dfrac{g}{l}\cdot\sin(\theta) + \dfrac{1}{ml^2}\cdot u\end{bmatrix}; \quad h\left(\begin{bmatrix}\theta\\ \dot{\theta}\end{bmatrix}\right) = [\theta]$$

```python
class SymbolicPendulum(SymbolicDynamicalSystem):
    def __init__(
        self, m: float = 1.0, l: float = 1.0, beta: float = 1.0, g: float = 9.81
    ):
        super().__init__()
        self.order = 1
        self.define_system(m, l, beta, g)

    def define_system(self, m_val, l_val, beta_val, g_val):
        theta, theta_dot = sp.symbols("theta theta_dot", real=True)
        u = sp.symbols("u", real=True)
        m, l, beta, g = sp.symbols("m l beta g", real=True, positive=True)

        self.parameters = {m: m_val, l: l_val, beta: beta_val, g: g_val}
        self.state_vars = [theta, theta_dot]
        self.control_vars = [u]
        self.output_vars = [theta]

        ml2 = m * l * l
        self._f_sym = sp.Matrix(
            [theta_dot, (-beta / ml2) * theta_dot + (g / l) * sp.sin(theta) + u / ml2]
        )
        self._h_sym = sp.Matrix([theta])
```

Approach: System Methods

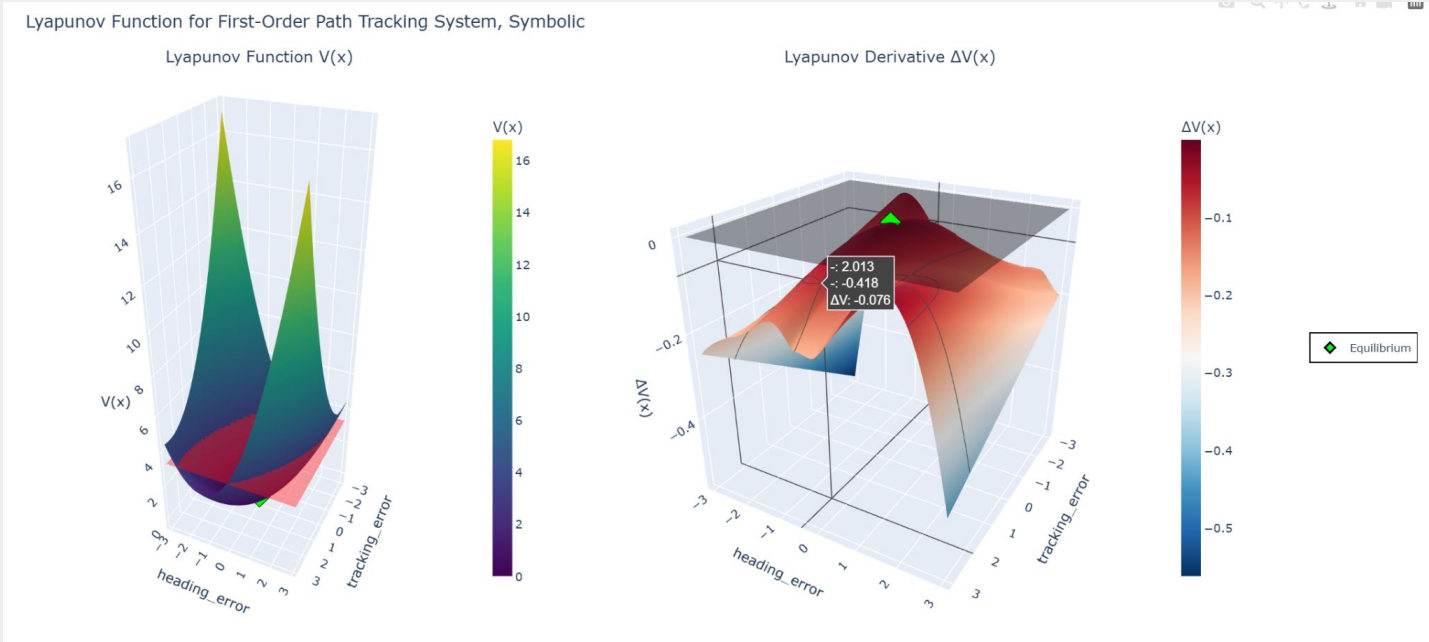
- SymbolicDynamicalSystem public methods exist to
  - Symbolically and numerically evaluate state and output dynamics as well as linearizations around pre-specified equilibrium
  - Create callable NumPy or PyTorch functions for downstream applications
  - Assess numerical and dynamical stability of the system
  - Calculate continuous time LQR and Kalman gain around equilibrium

Approach: System Methods

- Mainly designed to be backward compatible with the DiscreteTimeSystem class used in the code base, however
- GenericDiscreteTimeSystem public methods exist to
  - Simulate trajectories with or without control using numerical integration methods
  - Calculate discrete time LQR and Kalman gain around equilibrium
  - Plot system trajectories and phase portraits as interactive HTML files

Approach: Lyapunov Function Plotting

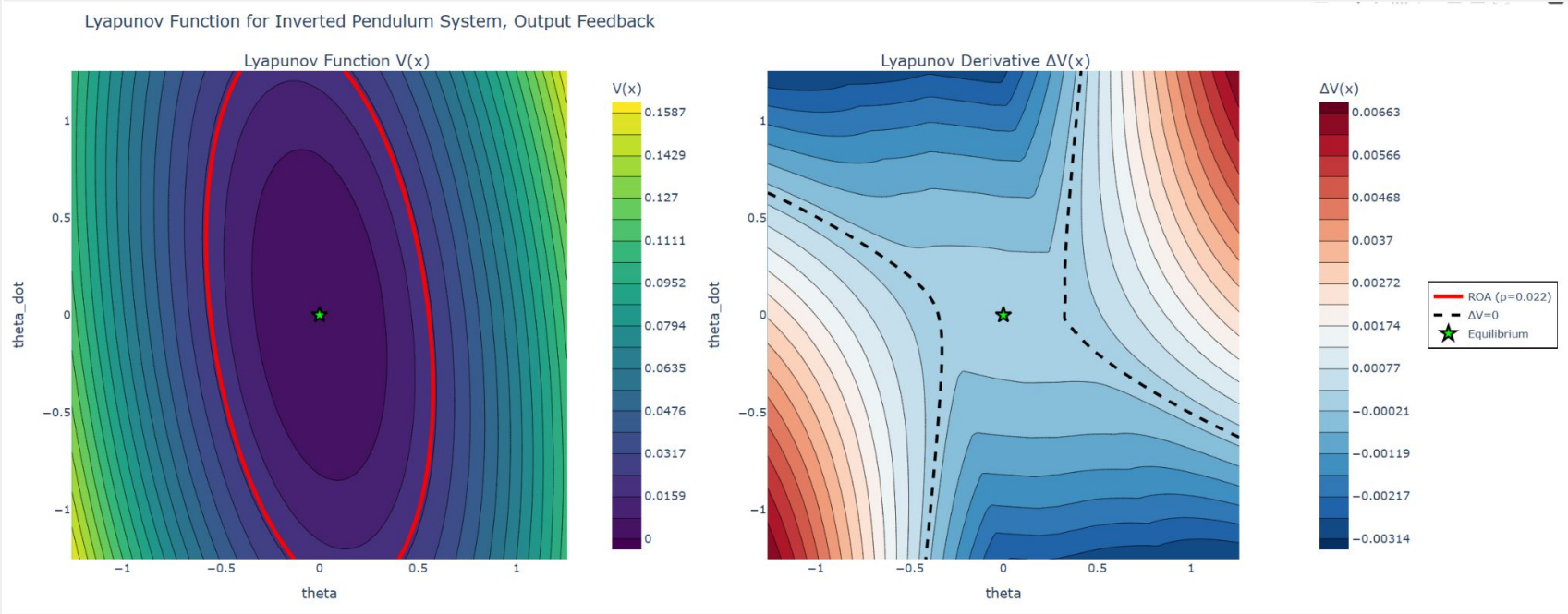
- Created functions for 2D and 3D plotting of resulting Lyapunov candidates and the closed loop derivative as interactive HTML files



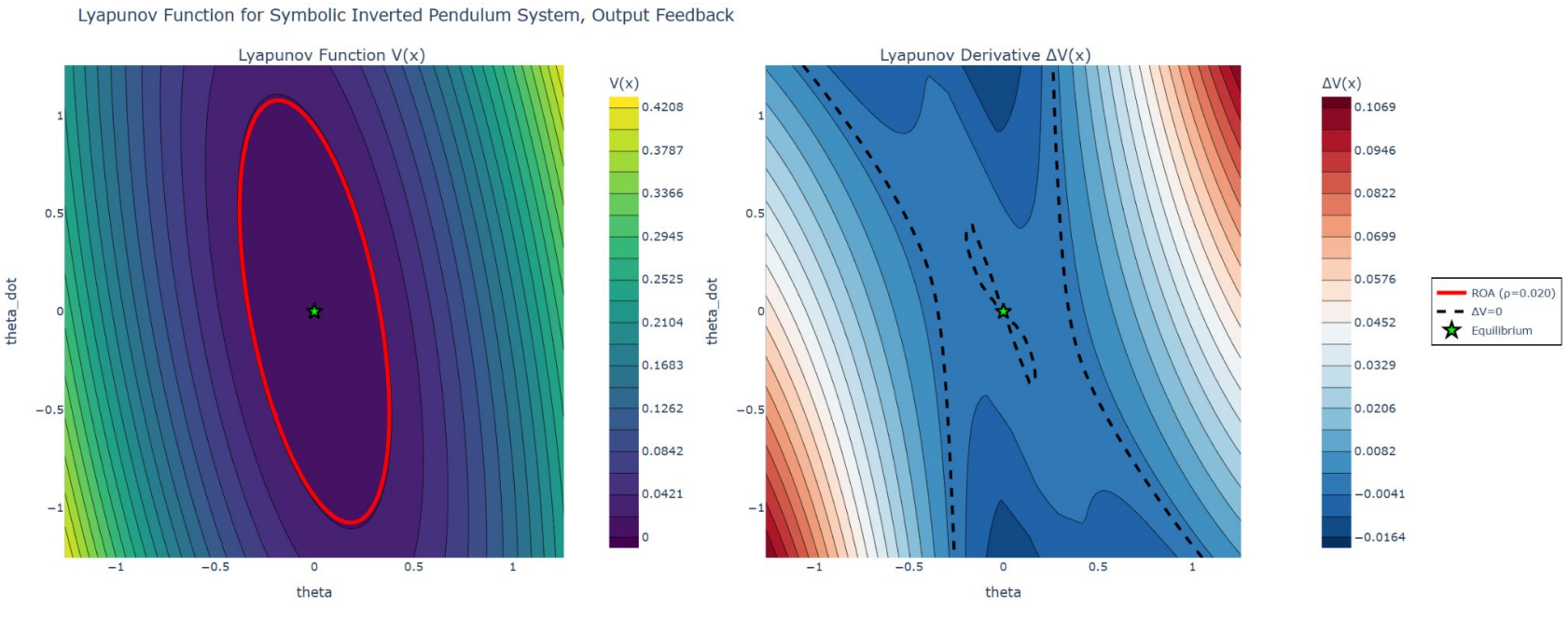
Approach: ROA Measurement

- ROA size measurement using Quasi-Monte Carlo Numerical Integration
  - Pick a number of states quasi-randomly
  - Evaluate the Lyapunov candidate and Lyapunov derivative at each sampled state
  - Ratio of states with Lyapunov candidate value above threshold compared to entire set of samples gives relative fraction of domain covered by ROA
  - If derivative values are greater than zero in some area of ROA, invalidates Lyapunov candidate

Result: Hard-Coded Pendulum



Result: Symbolic Pendulum



- Qualitative difference necessitated empirical testing

Results: Numerical Comparison

Metric	Author Hard-Coded System	Our Symbolic System
ρ	0.022181 ± 0.007968	0.023006 ± 0.003195
Fraction of $\mathcal{B}$ with $V(\xi) \leq \rho$	0.1781 ± 0.0714	0.174 ± 0.0477
Fraction of $\mathcal{B}$ with $\Delta V(\xi) \leq 0$	0.4977 ± 0.0554	0.4237 ± 0.0672
Fraction of $\mathcal{B}$ in Intersection	0.1781 ± 0.072	0.1736 ± 0.0522
Fraction of $\mathcal{S}$ where $\Delta V(\xi) > 0$	0.0131 ± 0.0131	0.001798 ± 0.001798

Table 1: Pendulum Output Feedback Metrics, Median $\pm$ Median Absolute Deviation, $N = 25$

- Both system types perform similarly
- The training algorithm does not guarantee a verified control Lyapunov function is found
- Lyapunov function used in this case was not a neural network but a learnable quadratic function

Results: Path-Tracking Numerical Comparison

Metric	Author Hard-Coded System	Our Symbolic System
ρ	3.423426 ± 0.183006	3.329456 ± 0.102035
Fraction of $\mathcal{B}$ with $V(\xi) \leq \rho$	0.6072 ± 0.003	0.6103 ± 0.0031
Fraction of $\mathcal{B}$ with $\Delta V(\xi) \leq 0$	1 ± 0	1 ± 0
Fraction of $\mathcal{B}$ in Intersection	0.6072 ± 0.003	0.6103 ± 0.0031
Fraction of $\mathcal{S}$ where $\Delta V(\xi) > 0$	0 ± 0	0 ± 0

Table 2: Path Tracking State Feedback Metrics, Median $\pm$ Median Absolute Deviation, $N = 25$

- Lyapunov function used in this case was a neural network
  - Demonstrates possibility to set algorithm hyperparameters such that valid Lyapunov functions are always found
- Difference is in dynamical system stability and hyperparameters
  - Path tracking system has a stable equilibrium whereas the inverted pendulum is unstable

Result: Failure on New Systems

- Ultimate goal was to enable arbitrary dynamical system specification for Lyapunov candidate generation
- We tried to implement two dynamical systems evaluated by authors but without example files for candidate training
  - CartPole and Planar Vertical Takeoff and Landing (PVTOL)
- Training did not converge
  - Likely due to inadequate hyperparameter tuning
  - Training is a bi-level optimization problem with competing objective functions

Conclusion: What We Did

- Create a framework for arbitrary dynamical system specification and simulation
- Construct functions for
  - Plotting Lyapunov candidates and associated derivative functions
  - Measuring ROA volume
- Integrated these capabilities into an existing SotA framework for neural Lyapunov function and controller synthesis
- Empirically compared Lyapunov candidate and controller synthesis of author’s code to our symbolic systems

Conclusion: What Next

- Finish README
- Develop better, more intuitive methods for setting training hyperparameters
  - Makes or breaks training
- Demonstrate Lyapunov candidate generation and controller synthesis on novel dynamical systems
- Assess integration into accompanying SotA algorithm for Lyapunov candidate and controller verification
  - Current code only yields candidates
- Adapt code further to better support partially observable systems and multiple equilibria
- Explore possibility of integrating visualizations with paper on Generalized Lyapunov Functions with RL from NeurIPS this year (paper, code)

Thank you

Backup Slides

Algorithms

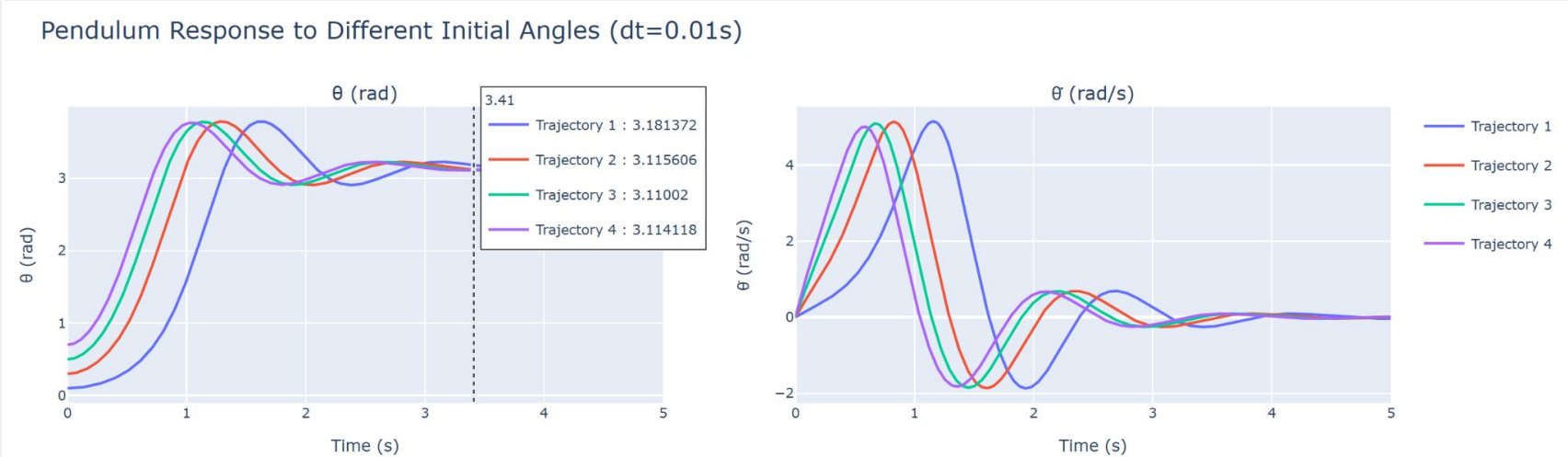
Algorithm 1 Lyapunov-stable Neural Control Verification

Input: neural-network controller π, observer network ϕ_{obs}, Lyapunov function V, sublevel set value estimate $\hat{\rho}_{\text{max}}$ (possibly from training), scaling factor λ, convergence tolerance tol
Output: certified sublevel set value ρ_{max}
// Find initial bounds ρ_{lo} and ρ_{up} for bisection
Verify (14) with $(\pi, \phi_{\text{obs}}, V, \hat{\rho}_{\text{max}})$ via α,β-CROWN
if verified **then**
    $\rho_{\text{lo}} = \hat{\rho}_{\text{max}}$
    $\rho_{\text{up}} =$ multiply $\hat{\rho}_{\text{max}}$ by λ until verification fails
else
    $\rho_{\text{up}} = \hat{\rho}_{\text{max}}$
    $\rho_{\text{lo}} =$ divide $\hat{\rho}_{\text{max}}$ by λ until verification succeeds
end if
// Bisection to find ρ_{max}
while $\rho_{\text{up}} - \rho_{\text{lo}} > tol$ **do**
    $\rho_{\text{max}} \leftarrow \frac{\rho_{\text{lo}}+\rho_{\text{up}}}{2}$
    Verify (14) with $(\pi, \phi_{\text{obs}}, V, \rho_{\text{max}})$ via α,β-CROWN
    if verified **then**
        $\rho_{\text{lo}} \leftarrow \rho_{\text{max}}$
    else
        $\rho_{\text{up}} \leftarrow \rho_{\text{max}}$
    end if
end while

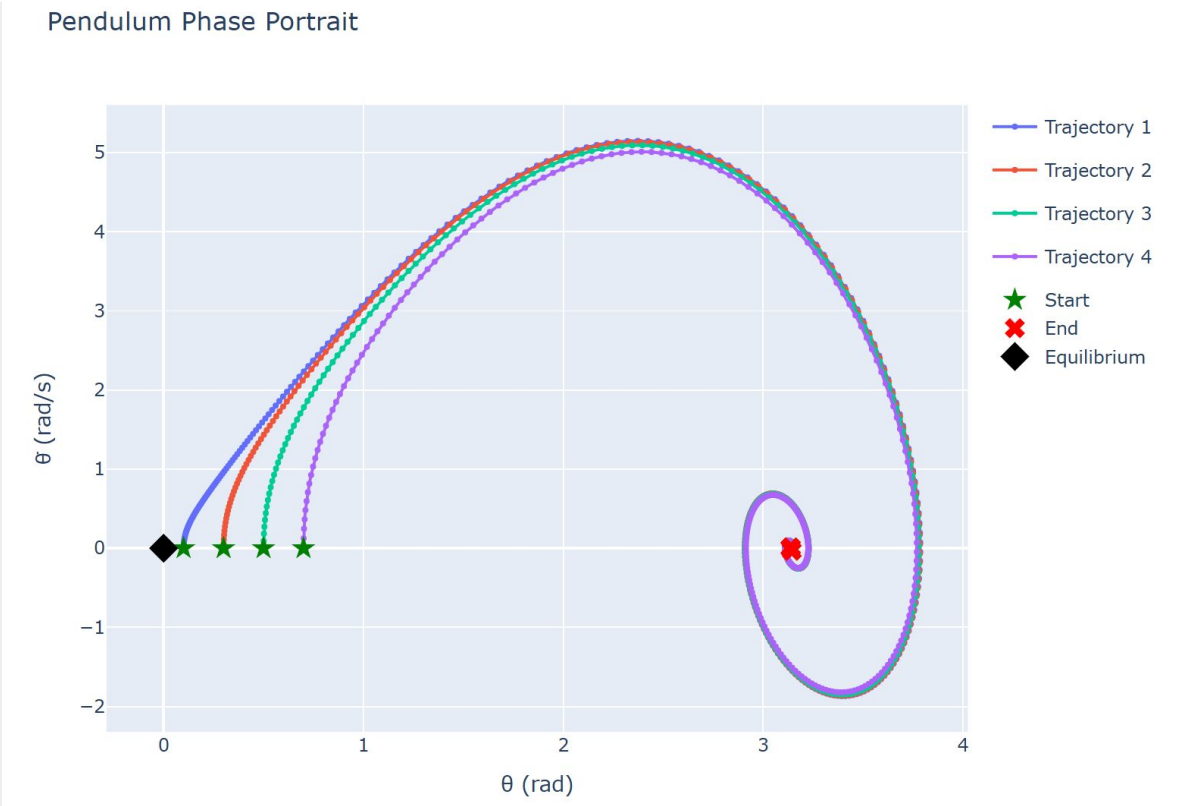
Algorithm 2 Training Lyapunov-stable Neural Controllers

Input: plant dynamics f and h, region-of-interest $\mathcal{B}$, scaling factor γ, PGD stepsizes α and β, learning rate η
Output: Lyapunov candidate V, controller π, observer ϕ_{obs} all in θ
Training dataset $\mathcal{D} = \varnothing$
for $iter = 1, 2, \cdots$ **do**
    Sample points $\bar{\xi}_j \in \partial\mathcal{B}$
    for $rho\_descent = 1, 2, \cdots$ **do**
        $\bar{\xi}_j \leftarrow \text{Project}_{\partial\mathcal{B}}\left(\bar{\xi}_j - \alpha \cdot \frac{\partial V(\bar{\xi}_j)}{\partial \bar{\xi}}\right)$
    end for
    $\rho = \gamma \cdot \min_j V(\bar{\xi}_j)$
    Sample counterexamples $\xi^i_{\text{adv}} \in \mathcal{B}$
    for $adv\_descent = 1, 2, \cdots$ **do**
        $\xi^i_{\text{adv}} \leftarrow \text{Project}_{\mathcal{B}}\left(\xi^i_{\text{adv}} + \beta \cdot \frac{\partial L_{\dot{V}}(\xi^i_{\text{adv}};\rho)}{\partial \xi_{\text{adv}}}\right)$
    end for
    $\mathcal{D} \leftarrow \{\xi^i_{\text{adv}}\} \cup \mathcal{D}$
    for $epoch = 1, 2, \cdots$ **do**
        $\theta \leftarrow \theta - \eta \nabla_\theta L(\theta; \mathcal{D}, \rho)$
    end for
end for

Approach: Example Trajectory Plot

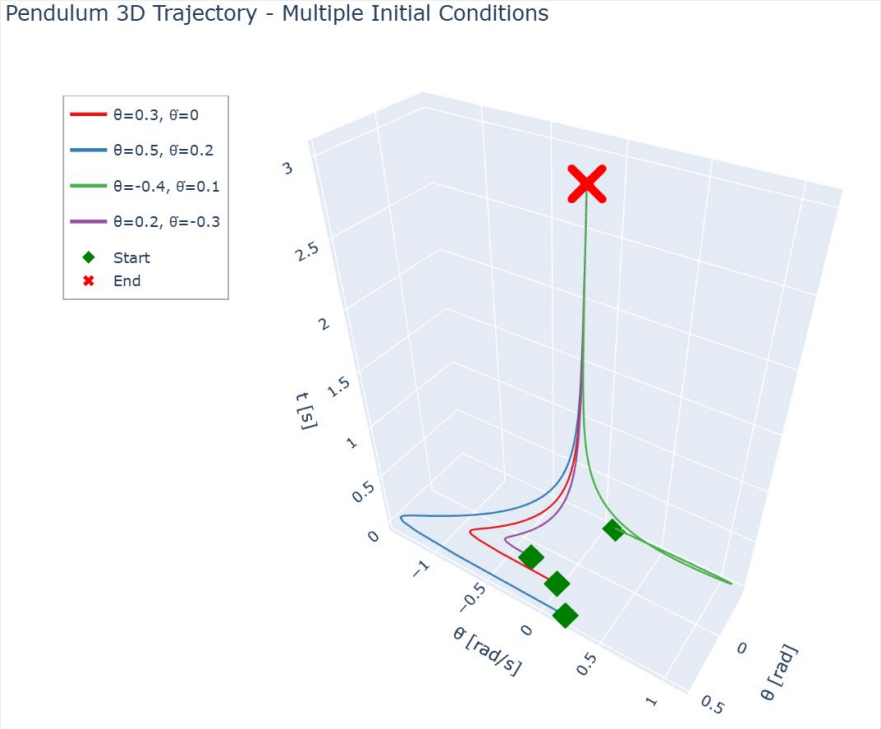


Approach Example Phase Portrait

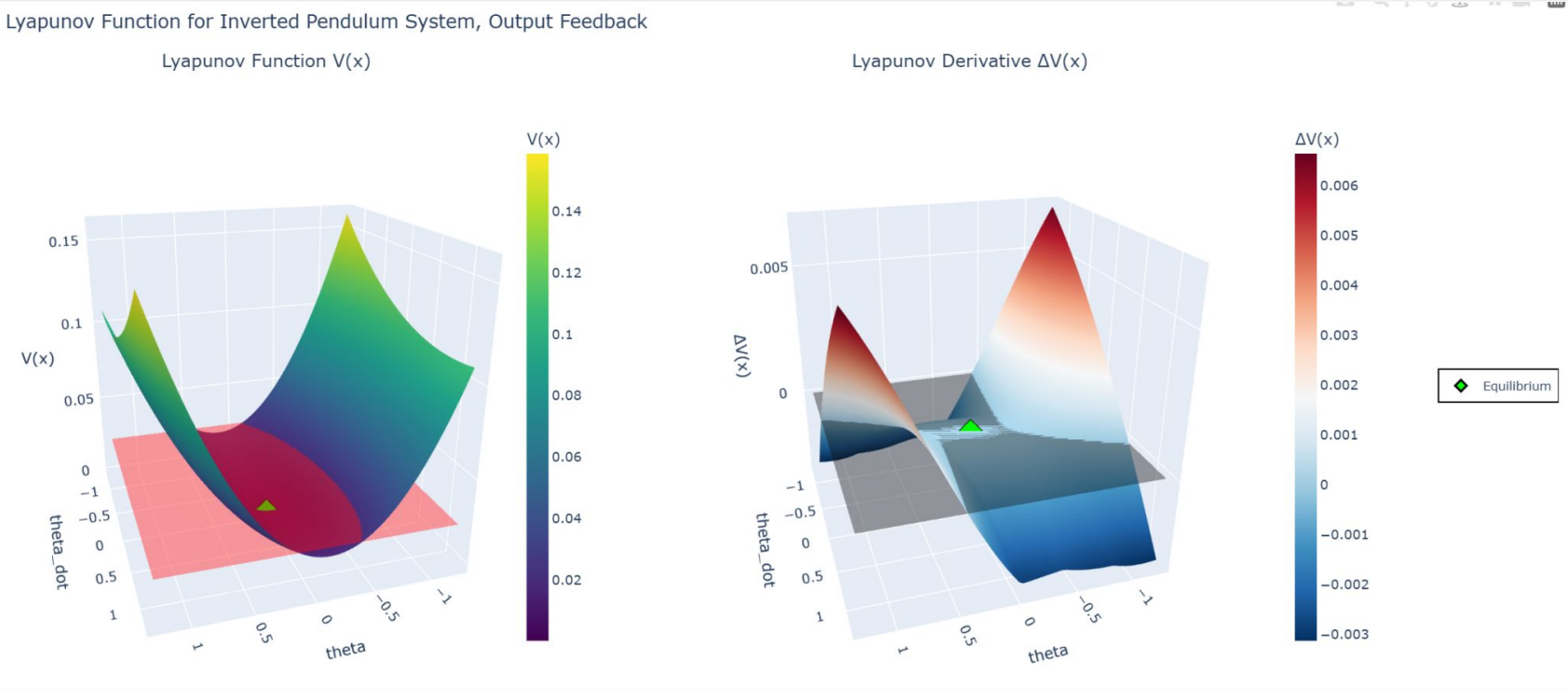


Example 3D Trajectory Plot

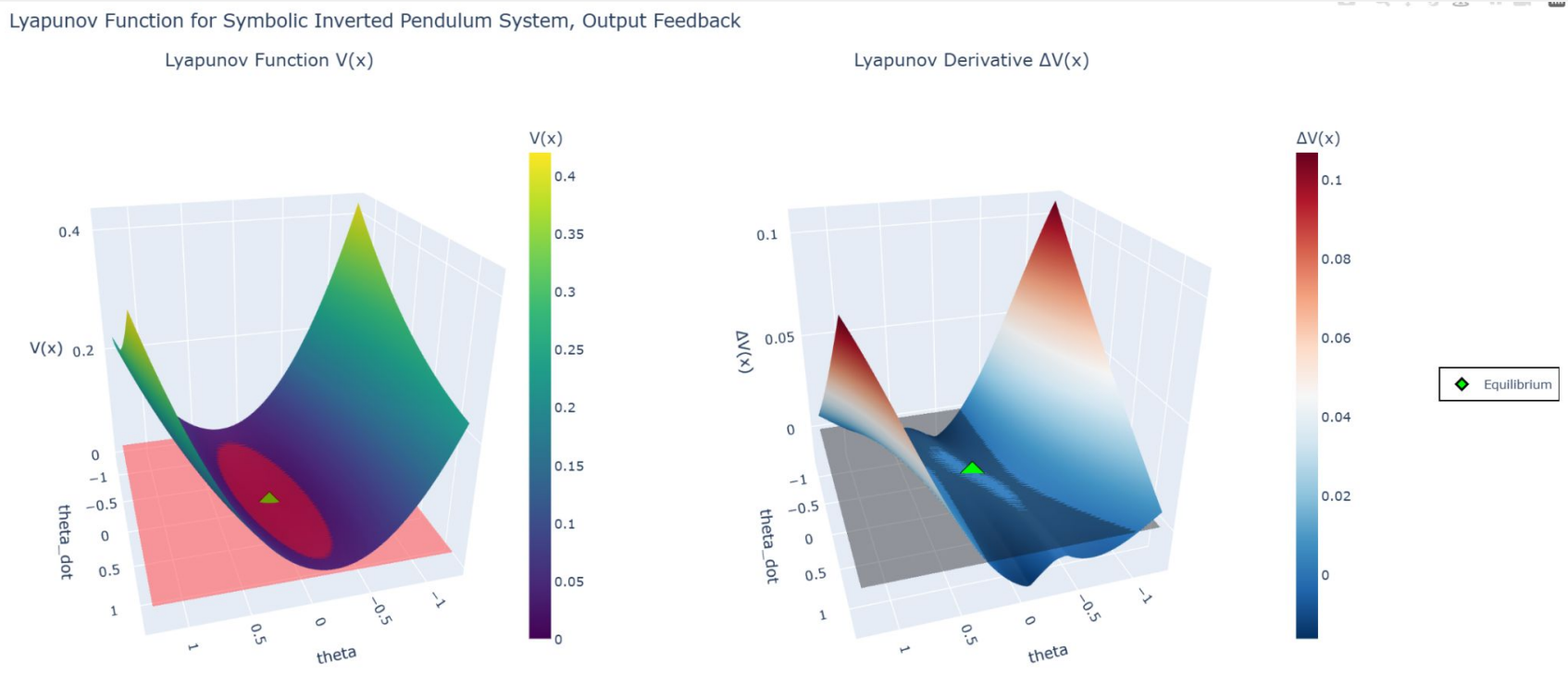
Inverted Pendulum with LQR control



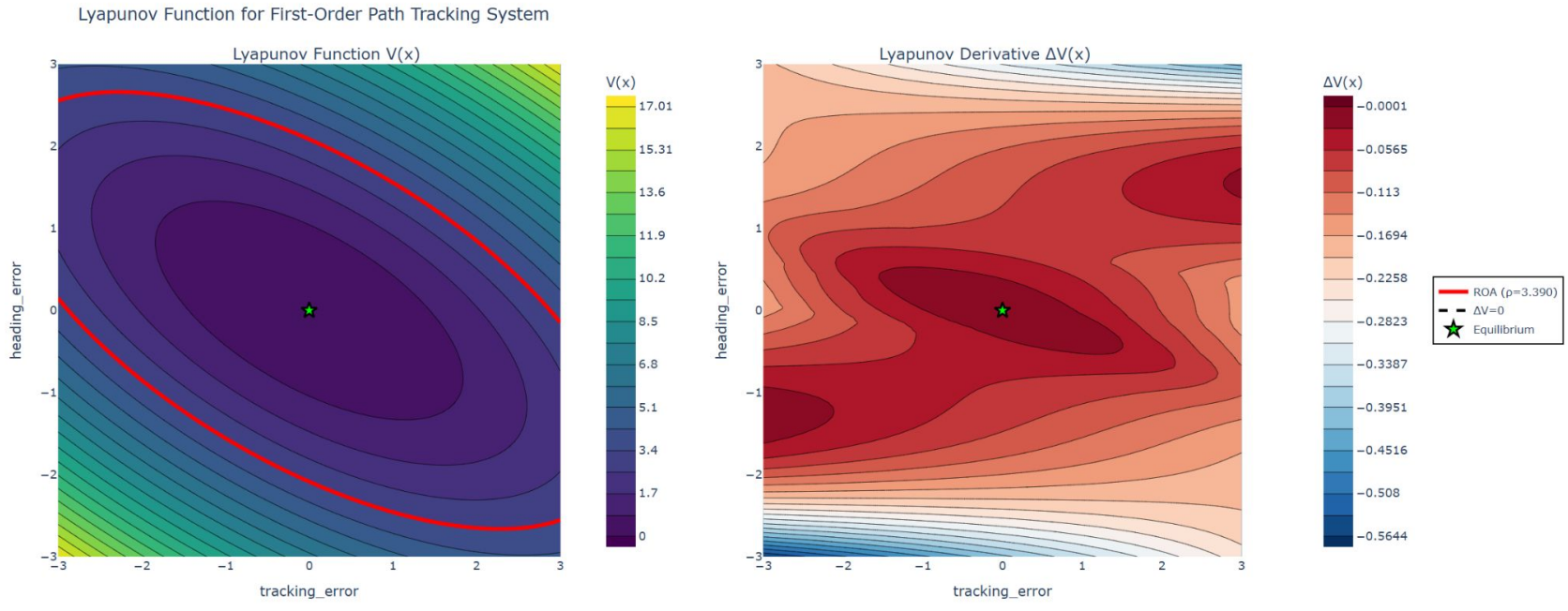
3D Plot, Hard-Coded System



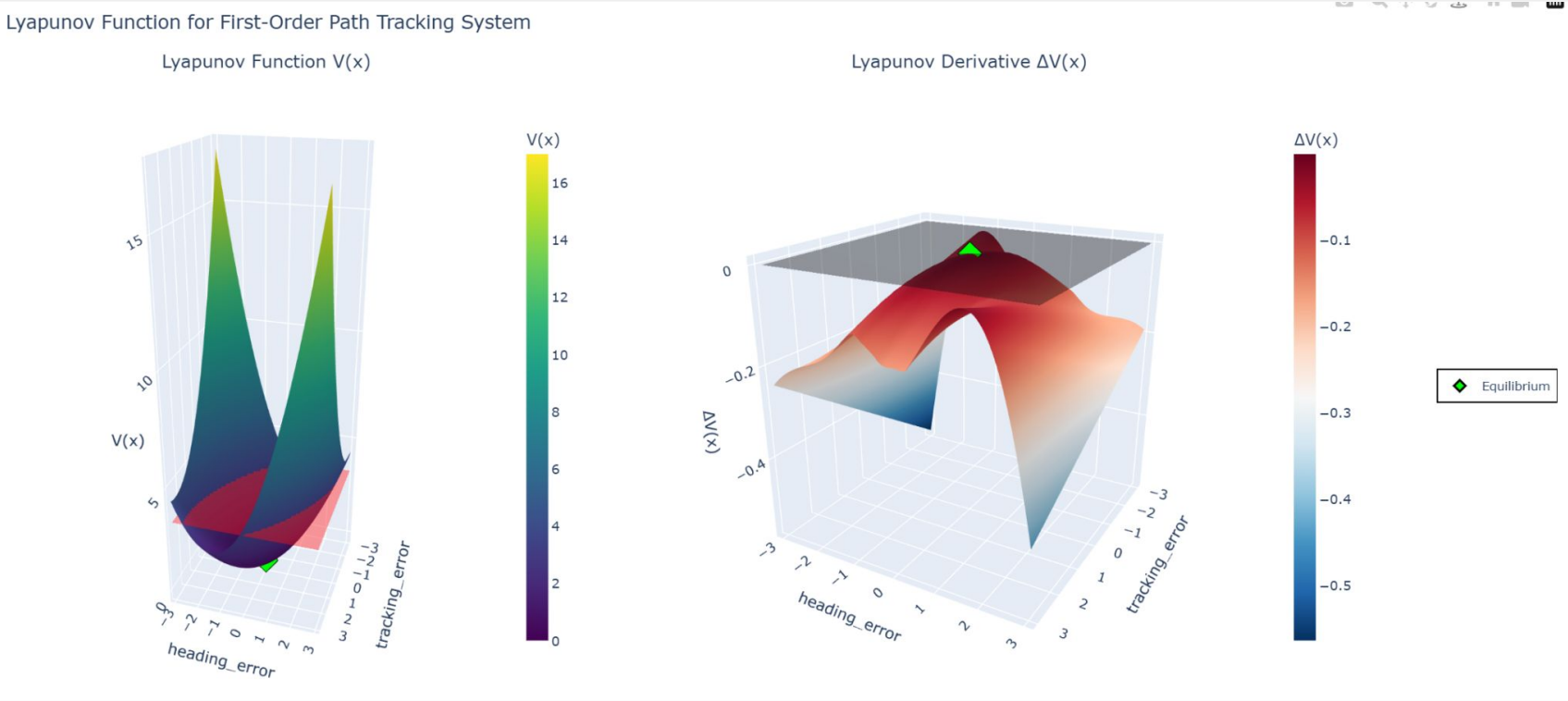
3D Plot, Symbolic System



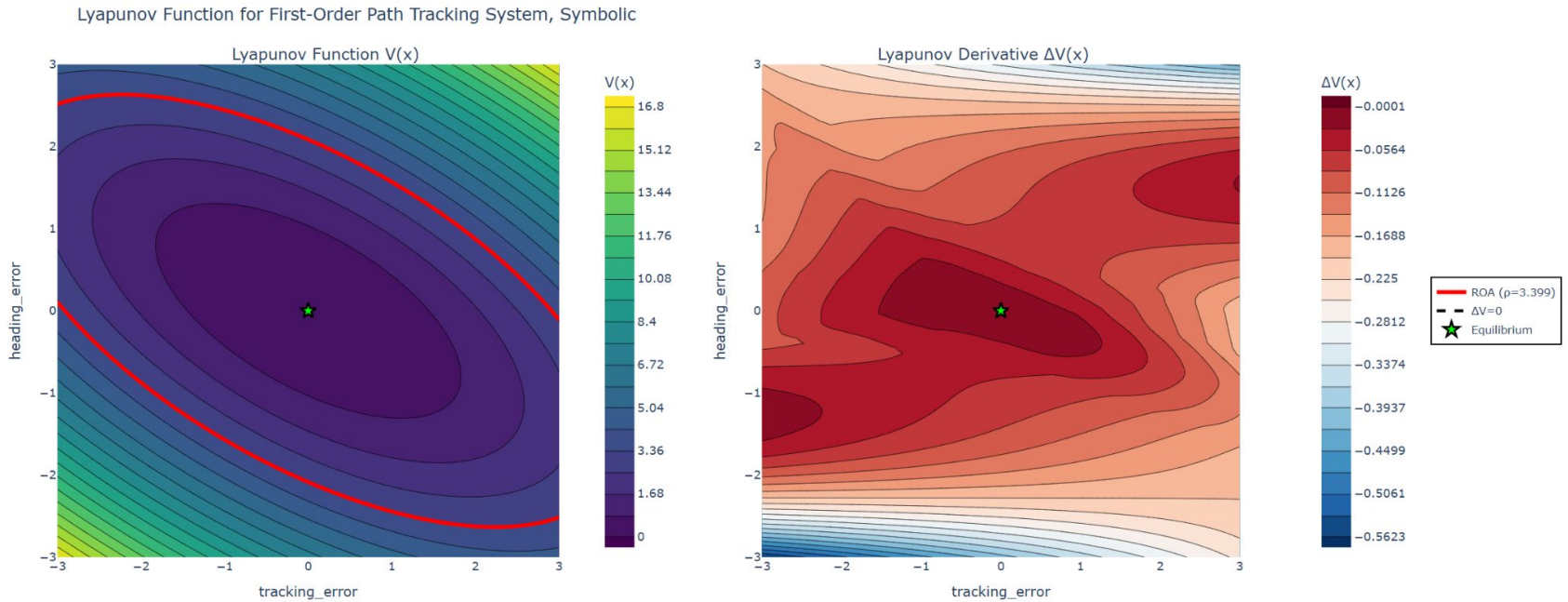
Path Tracking State Feedback, Hard-Coded, Default Hyperparameters



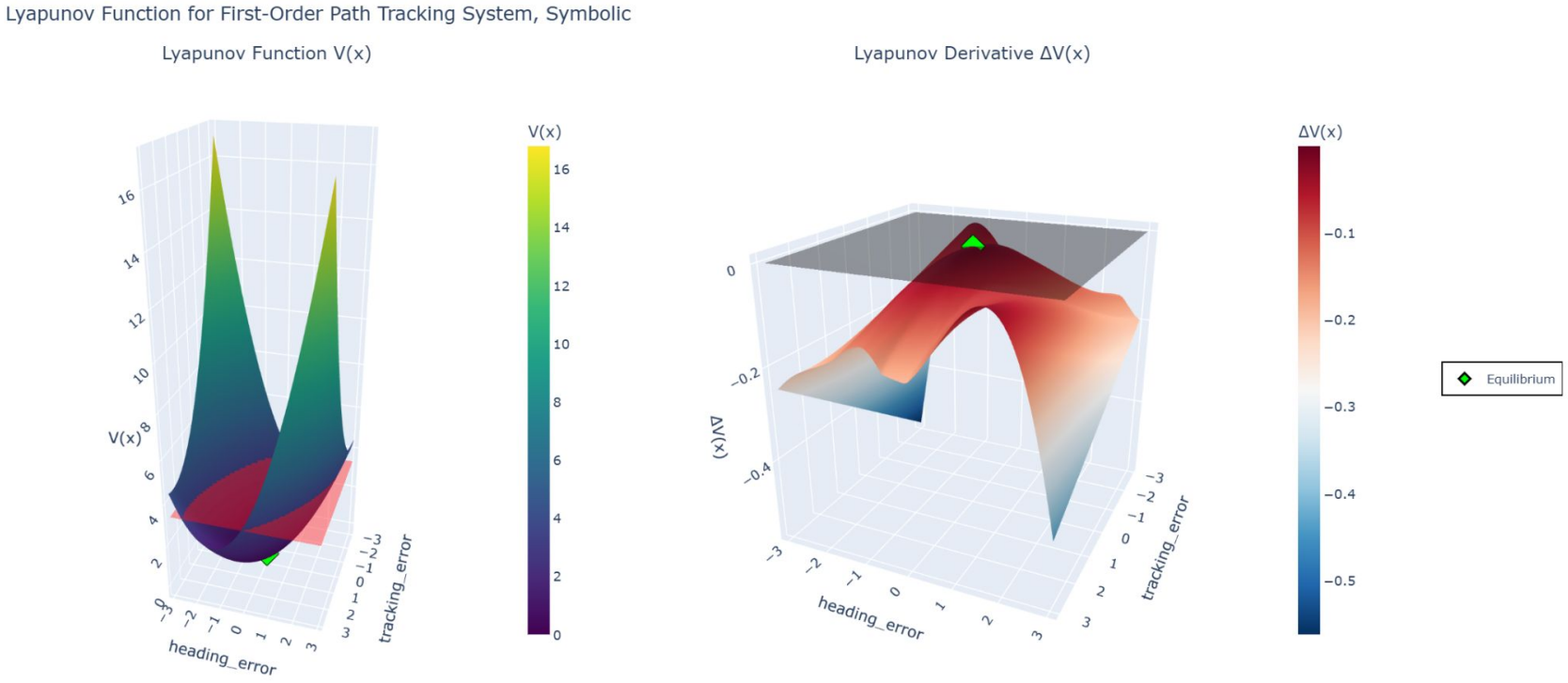
Path Tracking State Feedback, Hard-Coded, Default Hyperparameters



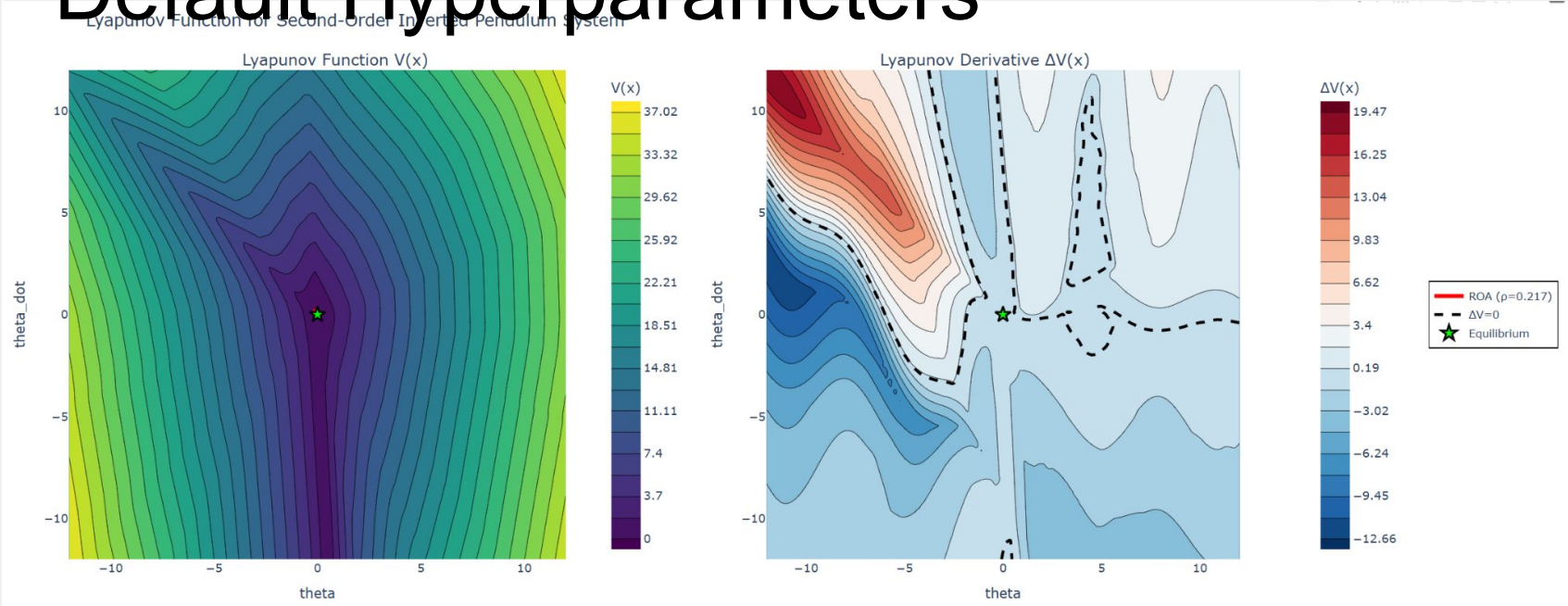
Path Tracking State Feedback, Symbolic, Default Hyperparameters



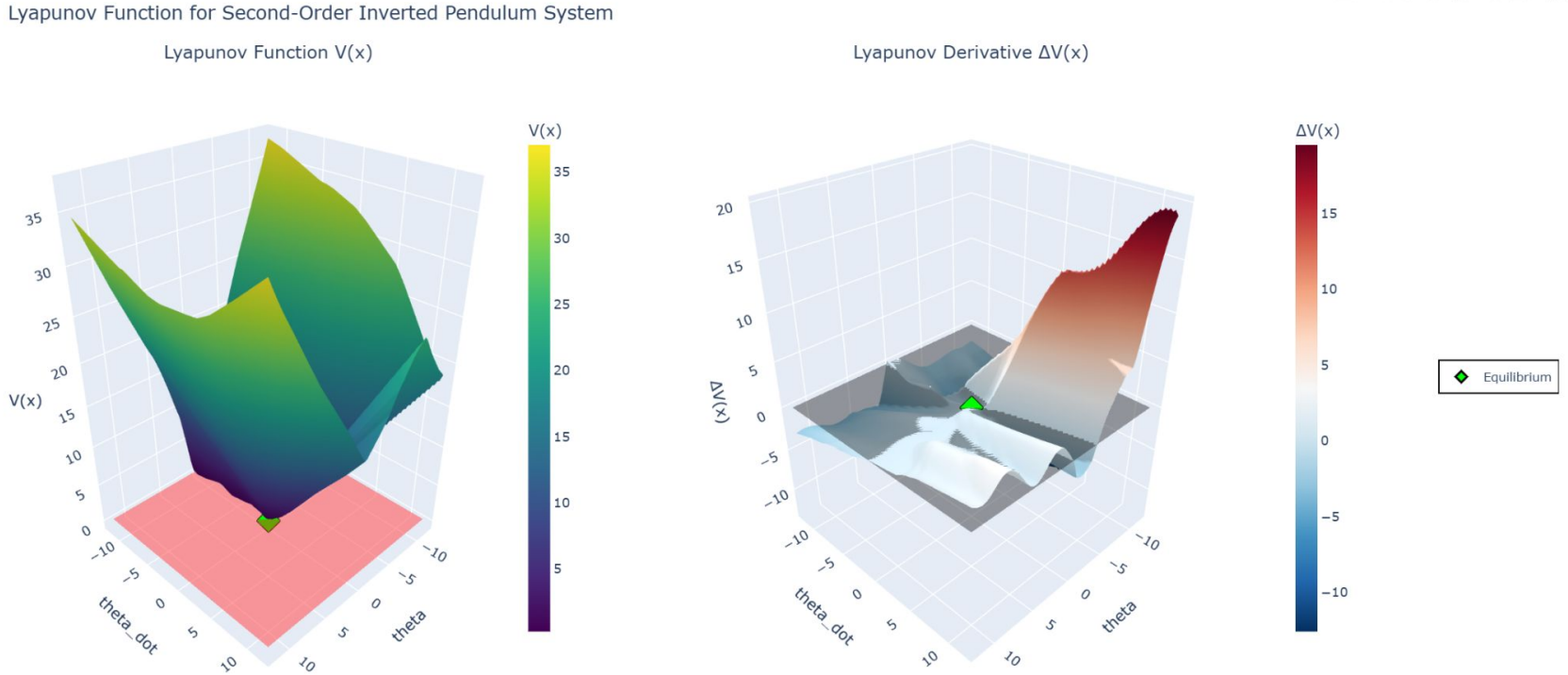
Path Tracking State Feedback, Symbolic, Default Hyperparameters



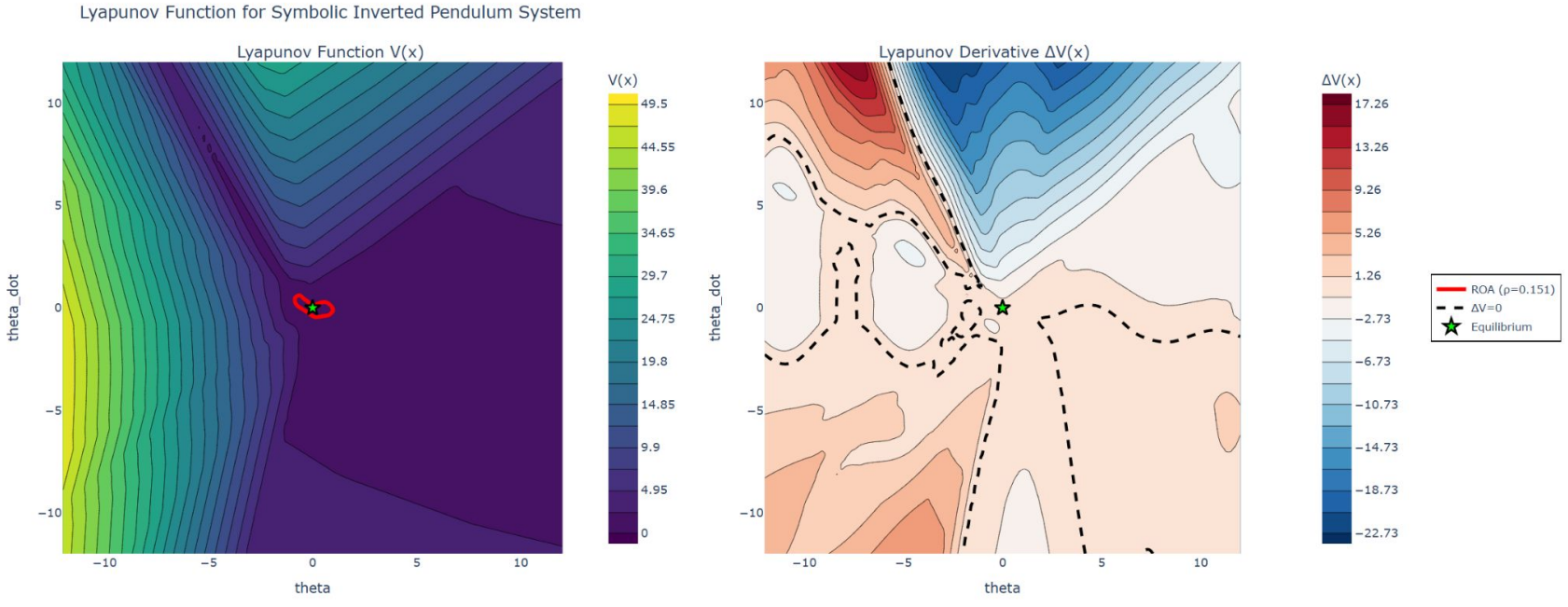
Pendulum State Feedback, Hard-Coded, Default Hyperparameters



Pendulum State Feedback, Hard-Coded, Default Hyperparameters



Pendulum State Feedback, Symbolic, Default Hyperparameters



Pendulum State Feedback, Symbolic, Default Hyperparameters